

What Are You Running? PuppetDB Knows.

Rich Burroughs, Daniel Parks

Puppet SRE

<http://bit.ly/2uetA88>

**We're the team at Puppet that
runs Puppet**

PuppetDB

Node Inventory

`web1.example.com`

`ipaddress => 10.0.0.1`
`osfamily => Debian`

Resources

`Package[bash]`

`ensure => '3.2.57'`

`web1.example.com`

How do I get it?

- **Included with Puppet Enterprise**
- **Open source users can use the puppetlabs/puppetdb module to install and manage it**

Puppet Query Language (PQL)

Node Inventory

fact	value
certname	web1-prod.example.com
ipaddress	10.0.0.2
os.family	Debian

```
# Get information about nodes (hosts)
inventory {

    # Only nodes where the certname (FQDN) contains "prod"
    certname ~ "prod"

    # and the OS is Debian
    and facts.os.name = "Debian"
}
```



```
$ puppet query 'inventory { certname ~ "prod"
                           and facts.os.name = "Debian" }'

[
  {
    "certname": "web1-prod.example.com",
    "timestamp": "2017-03-22T19:36:20.095Z",
    "facts": {
      "ipaddress": "10.0.0.2",
      "memoryfreeinbytes": "1766612992",
      "os": {
        "name": "Debian",
        . . .
      }
    }
  }
]
```

Resources

```
class profile::database ( $password ) {  
  # This is a resource:  
  postgresql::server::db { 'myapp_database':  
    user      => 'myapp',  
    password => $password,  
  }  
}
```

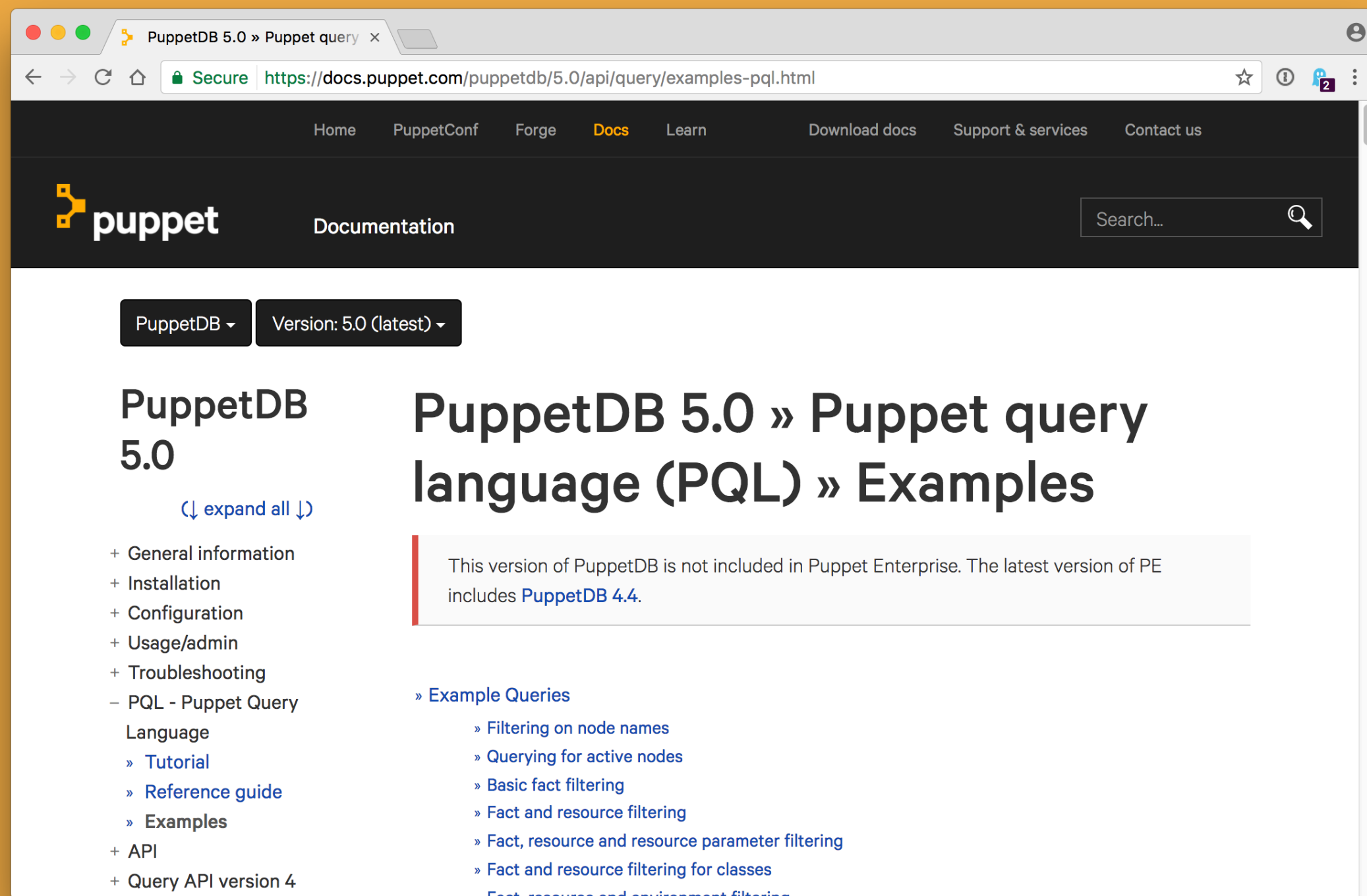


```
$ puppet query 'resources { type = "Postgresql::Server::Db" }'
[
  {
    "certname": "db1-prod.example.com",
    "file": ". . ./profile/manifests/database.pp",
    "line": 3,
    "title": "myapp_database",
    "parameters": {
      "user": "myapp",
      "grant": "ALL",
      . . .
    }
  }
]
```

Returns file and line number
where the resource is defined
^ Includes all parameters for
the resource, including the ones
that aren't directly specified
and come from defaults or hiera

```
$ puppet query 'resources { type = "Postgresql::Server::Db" }'
[
  {
    "certname": "db1-prod.example.com",
    "file": ". . ./profile/manifests/database.pp",
    "line": 3,
    "title": "myapp_database",
    "parameters": {
      "user": "myapp",
      "grant": "ALL",
      . . .
    }
  }
]
```

```
$ puppet query 'resources[certname,title]
{ type = "Class" and title ~ "Role::" }'
[
  {
    "certname": "web1-prod.example.com",
    "title": "Role::Web"
  },
  {
    "certname": "db1-prod.example.com",
    "title": "Role::Db"
  },
  . . .
]
```

I like the PQL examples page

Queries in Puppet Code

We can also PQL in Puppet code. We often use this instead of exported resources.

```
# Find load balancer members
puppetdb_query('inventory { certname ~ "^web.*-prod" }').each |$node| {
  haproxy::balancermember { $node['facts']['fqdn']:
    listening_service => 'www',
  }
}

# Create icinga2 host objects for all nodes in inventory
puppetdb_query('inventory {}').each |$node| {
  icinga2::object::host { $node['certname']:
    'ipv4_address' => $node['facts']['ipaddress'],
  }
}
```

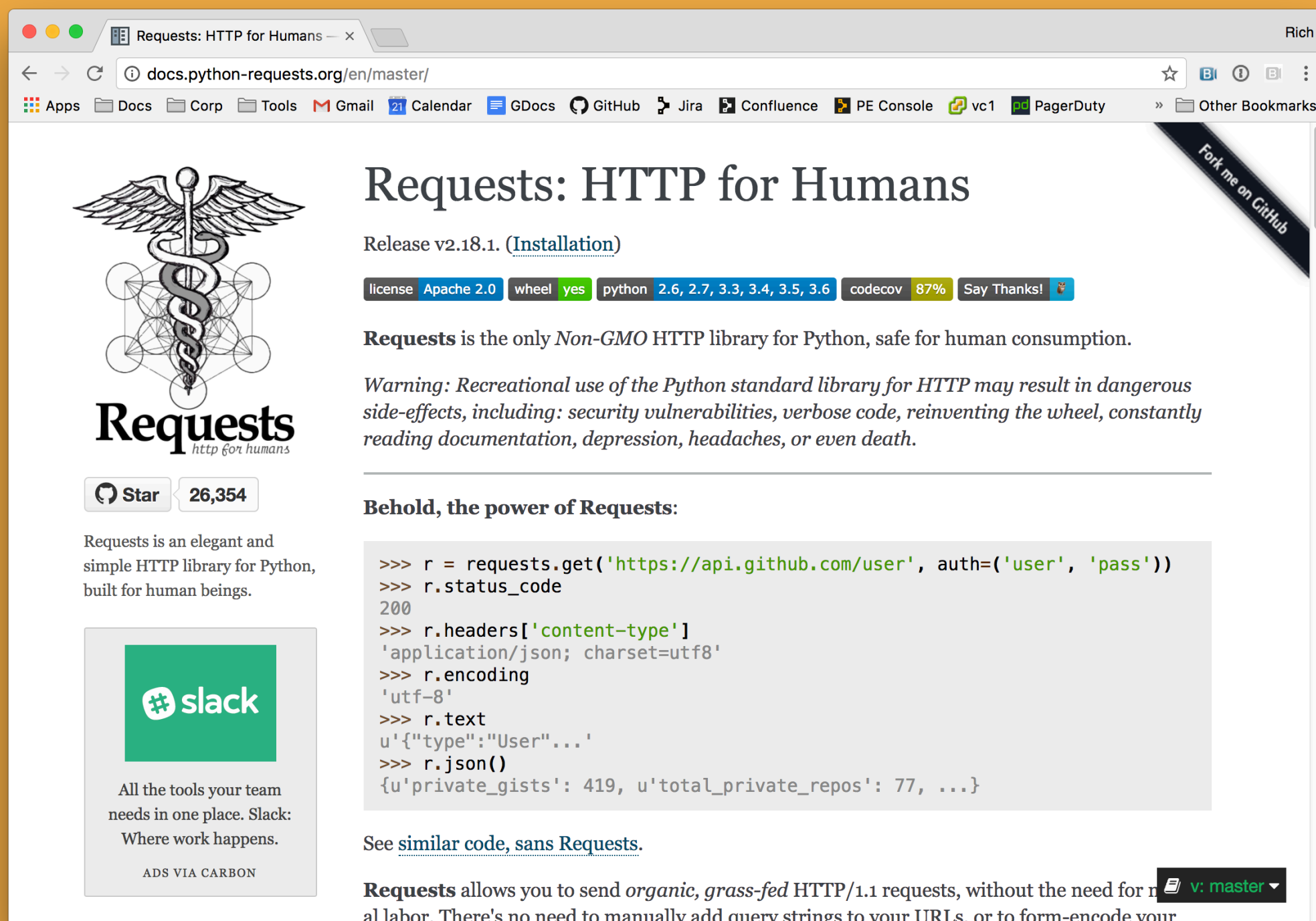
Here's an example of setting up hosts in our Icinga2 monitoring system.

^ We query for all nodes and create a host resource for them based on facts defined for each node.

REST API

Can access via HTTP or
HTTPS

^ If you use as part of PE, you
can use RBAC auth tokens



Python's Requests library is great
^ Can also use libraries for other languages that do HTTP, like Ruby's rest-client


```
>>> import requests
>>> url = "http://httpbin.org/ip"
>>> r = requests.get(url)
>>> print(r.text)
{
  "origin": "10.0.0.1"
}
```

```
def get_nodes():  
    nodes = []  
    url = "http://localhost:8080/pdb/query/v4/nodes"  
    r = requests.get(url)
```

```
def get_nodes():  
    nodes = []  
    url = "http://localhost:8080/pdb/query/v4/nodes"  
    r = requests.get(url)  
    response = json.loads(r.text)  
    for i in response:  
        nodes.append(i['certname'])  
    return nodes
```




Rich Burroughs
@richburroughs



Daniel Parks
@daemonhoarse

Slides on GitHub
<http://bit.ly/2uetA88>